

SpecFirst Phase 4

Freeze Spec: Complete Architecture and Implementation Guide

Purpose: convert a supported classified component and its seven-layer manifest into a deterministic, hashed, component-specific accessibility contract.

Current artifact	Value
Component	PetCombobox
Pattern	react-select-only-combobox
Manifest	react-select-only-combobox v1.0.0
Locked spec status	locked
Generated checks	11
Hash algorithm	sha256

This document describes what Phase 4 does, what it must not do, its input/output contracts, how it specializes manifest requirements, how hashing works, and what Phase 5 can rely on.

1. Phase 4 in one sentence

Phase 4 takes the generic seven-layer rulebook selected in Phase 3 and freezes it into a component-specific, hashed accessibility contract that Phase 5 will convert into executable tests.

Pipeline context:

```
Phase 1: Analyze component facts
Phase 2: Classify UI pattern
Phase 3: Load and validate rule manifest
Phase 4: Freeze component-specific spec
Phase 5: Generate tests
Phase 6: Run baseline tests
Phase 7: Bob patches component
Phase 8: Verify and report
```

The central design rule is that Phase 4 defines the target before Bob appears. Bob must never define, edit, or weaken the success criteria.

2. Scope and non-scope

What Phase 4 does

- Reads one completed run folder and validates Phase 1 to Phase 3 artifacts.
- Uses manifestUsed.json as the exact rulebook snapshot selected by Phase 3.
- Specializes generic manifest requirements into component-specific checks.
- Binds checks to the trigger, popup, option elements, control state, accessible-name strategy, and component scope.
- Copies manual review boundaries, Bob patch constraints, and report-boundary language.
- Computes a stable SHA-256 hash over the locked contract.
- Writes lockedSpec.json and allows Phase 5 to continue only if the contract is valid.

What Phase 4 must not do

- Does not call Bob.
- Does not generate Playwright tests.
- Does not run tests or render UI.
- Does not patch or modify the component.
- Does not inspect runtime DOM behavior.
- Does not claim accessibility or WCAG compliance.
- Does not re-resolve the manifest registry. It uses the Phase 3 snapshot.

3. Inputs

Phase 4 accepts either a run ID or a direct path to manifestLoadResult.json. Both resolve to the same run directory.

Input artifact	Role in Phase 4
componentAnalysis.json	Provides component facts: props, state, JSX element IDs, trigger/popup/options, source locations, snippets, accessible-name candidates, risk flags.
classification.json	Confirms supported pattern and manifestId. Phase 4 must skip if canContinue is false.
manifestLoadResult.json	Confirms the manifest was loaded and validated by Phase 3. Phase 4 must skip or fail if this is not loaded.
manifestUsed.json	The exact manifest snapshot used to generate the locked contract. This is the rulebook source of truth.

The current locked spec records SHA-256 hashes for all four upstream artifacts, making the contract traceable to exact prior inputs.

4. Gate behavior and status model

Status	Meaning	nextPhase
locked	All required inputs are valid, checks are generated, and hash computation succeeds.	canContinue true
skipped	Phase 2 or Phase 3 intentionally blocked continuation. This is a safe stop, not an error.	canContinue false
failed	Continuation was expected, but required artifacts, bindings, manifest content, or hashing are missing or inconsistent.	canContinue false

Hard failures include missing trigger, popup, options, manifest requirements, manual review boundary, report boundary, Bob constraints, pattern/manifest mismatch, or empty generated checks.

5. Output: lockedSpec.json

The locked spec is the only Phase 4 output. It is the source of truth for Phase 5 test generation and the contract Bob will later be asked to satisfy.

Top-level section	Purpose
schemaVersion, status	Identifies the artifact schema and whether the spec is locked, skipped, or failed.
sourceArtifacts	Records relative input paths and SHA-256 hashes for provenance.
generationPolicy	States that the phase is deterministic and did not call Bob, generate tests, render UI, patch code, or make compliance claims.
component	Copies the Phase 1 component identity and metadata.
classification	Copies the Phase 2 supported pattern summary.
manifest	Copies the Phase 3 manifest identity and seven-layer source model.
spec	Stores deterministic spec ID, createdAt, and createdBeforePatch flag.
componentBindings	Maps manifest concepts to actual element IDs, snippets, source locations, accessible-name binding, and state.
componentScope	Defines the wrapper selector Phase 5 should use for scoped testing and axe scans.
testHarnessHints	Hints for rendering the component in a controlled test route or harness.
checks	Component-specific checks generated from manifest requirements.

Top-level section	Purpose
manualReviewRequired	Manual review items carried forward to prevent overclaiming.
bobPatchConstraints	Constraints Phase 7 will include in the Bob remediation prompt.
accessibilityClaimBoundary	Allowed and forbidden accessibility language.
phaseStateBoundary	Process claims forbidden at Phase 4, such as code patched or tests generated.
integrity	Hash policy and final SHA-256 hash.
nextPhase	Handoff to Phase 5.

6. Component bindings

Component bindings are the heart of Phase 4. They turn abstract manifest requirements into concrete obligations for one component.

Binding	Current value	Why it matters
triggerElementId	jsx-3	The element that will become role=combobox and receive keyboard handling.
popupElementId	jsx-4	The conditional popup that should become role=listbox.
optionElementIds	jsx-5	Mapped option elements that should expose role=option and aria-selected.
controlState	open	The state that drives aria-expanded and popup visibility.
componentScope	[data-specfirst-root="PetCombobox"]	The scope selector Phase 5 should use for tests and axe scans.

Existing ARIA signals are context only. Phase 4 must not mark them as passing because no runtime or test evaluation occurs in this phase.

7. Check generation

For the MVP, the react-select-only-combobox manifest must produce the full set of 11 required checks. Phase 4 must fail if any required MVP check is missing.

Check ID	Strategy	Purpose
accessible-name	role-name-query	The combobox trigger exposes a non-empty accessible name that persists regardless of the current selection state.
combobox-role	role-query	The primary trigger element exposes role=combobox.
expanded-state	attribute-toggle	The trigger element exposes aria-expanded based on the component popup control state.
popup-associated	attribute-check	The trigger element is programmatically associated with the candidate popup.
popup-listbox	role-query	The popup element exposes role=listbox.
option-roles	role-query	Mapped candidate option elements expose option semantics.

Check ID	Strategy	Purpose
selected-state	attribute-check	The selected option exposes aria-selected=true; all other options expose aria-selected=false.
keyboard-arrow-down-opens	keyboard-open	ArrowDown on the trigger opens the popup and moves focus to the first option.
keyboard-enter-selects	keyboard-select	Enter on a focused option accepts that option, closes the popup, and returns focus to the trigger.
keyboard-escape-closes	keyboard-close	Escape closes the popup without changing the current selection and returns focus to the trigger.
axe-scan	axe-scan	axe-core scoped to the component root returns zero violations in both collapsed and expanded states.

Each check preserves seven-layer traceability through the layers field. This allows the evidence report to show where each requirement came from.

8. Focus management strategy

The current manifest and locked spec use an aria-activedescendant strategy for the select-only combobox. DOM focus remains on the trigger/combobox while the active option is referenced by aria-activedescendant.

This distinction matters for Phase 5. Generated keyboard tests should check active option state without assuming DOM focus moves to each option.

Field	Value
type	aria-activedescendant
domFocusRemainsOn	trigger
activeOptionReferencedBy	aria-activedescendant

9. Boundaries: claims and phase state

Phase 4 separates accessibility-claim limits from process-state limits.

Accessibility claims stay restricted throughout the pipeline. Phase-state claims are specific to Phase 4, because later phases may truthfully generate tests, patch code, and verify behavior.

Boundary	Examples
Allowed accessibility wording	
Forbidden accessibility wording	
Forbidden at Phase 4	

10. Manual review and Bob constraints

Manual review items are part of the contract. They prevent Phase 8 from overstating what automated checks can prove.

Manual review ID	Reason
screen-reader-announcement-quality	Automated tests confirm ARIA attributes are present; they cannot verify that NVDA, JAWS, or VoiceOver actually announce the combobox, options, and state changes in a usable way. Requires manual testing with at least one screen reader.
visual-focus-quality	Automated tests confirm focus moves to the correct element; they cannot verify that the visible focus indicator meets WCAG 2.4.11 or is perceptible to low-vision users. Requires manual visual inspection.
native-select-preference	Automated tests cannot determine whether a native element would better serve users in this product context. A human must evaluate whether the custom component is justified.
cross-browser-at-behavior	Playwright tests run in a single browser/AT pairing. Cross-browser screen reader behavior (e.g. Chrome+NVDA vs Safari+VoiceOver) must be verified manually.
option-label-meaningfulness	Automated tests confirm option text is non-empty; they cannot verify that option labels are meaningful, unambiguous, or appropriate in the product context.
cognitive-load	The number of options, grouping strategy, and interaction model may impose cognitive load. This requires human judgment and cannot be automated.

Bob patch constraints are carried forward so Phase 7 can generate a constrained remediation prompt.

- May only add or correct ARIA attributes defined in ariaValidity
- May only add keyboard event handlers defined in ariaPattern.keyboardModel
- Must not restructure the DOM or change element types
- Must not introduce ARIA roles not listed in ariaValidity
- Must flag invalid host elements for human review instead of patching silently
- Must not create a custom combobox where a native would suffice

11. Integrity and hashing

Phase 4 must make the contract tamper-evident. Later phases can re-hash the locked spec and detect if the contract changed after it was frozen.

```
hashAlgorithm: sha256
hashFieldPolicy: integrity.hash set to null before hashing
canonicalization: recursive-key-sort
hashIncludesCreatedAt: True
hash: sha256:2da6c4e5f996b59490aabdad3aca2042b9ed260fe25284329758898182bd7807
```

Hash procedure:

- 1 Build the complete lockedSpec object.
- 2 Set integrity.hash to null.
- 3 Recursively sort object keys to produce canonical JSON.
- 4 Stringify the canonical JSON.
- 5 Compute SHA-256 and write integrity.hash as sha256:.

12. Generation workflow

- 1 Resolve the run directory from the CLI input.
- 2 Load componentAnalysis.json, classification.json, manifestLoadResult.json, and manifestUsed.json.
- 3 Validate artifact schemas and cross-check component identity.
- 4 If Phase 2 or Phase 3 blocked continuation, write a skipped lockedSpec.json.
- 5 Confirm manifest ID matches classification.nextPhase.manifestId.
- 6 Extract component bindings from componentAnalysis.interactionModel.
- 7 Select the accessible-name binding and preserve existing name signals separately.
- 8 Generate one check per required manifest requirement, preserving testStrategy and layers.
- 9 Copy manual review, Bob constraints, and claim boundaries from the manifest and Phase 3.
- 10 Add componentScope, testHarnessHints, focus management, and source artifact hashes.
- 11 Hash and write lockedSpec.json.

13. Failure and skipped behavior

Case	Expected status	Reason
Phase 2 canContinue=false	skipped	The classifier intentionally blocked downstream work.
Phase 3 status not loaded	skipped or failed	Skip if intentional, fail if expected continuation but missing data.
manifestUsed.json missing	failed	Phase 4 must use the manifest snapshot selected by Phase 3.
trigger/popup/options missing	failed	Cannot bind requirements to a concrete component.
manualReviewRequired missing	failed	No safe boundary against overclaiming.
report boundary missing	failed	No safe language boundary.
Bob constraints missing	failed	Phase 7 would lack safe remediation constraints.
required MVP check missing	failed	Contract would be incomplete.

14. Test plan for Phase 4

Add test:spec with deterministic fixtures. Use a fixed createdAt in core tests so hashes are stable.

- Supported combobox run produces lockedSpec.json with status locked.
- All 11 required checks are generated and requiredMvpChecksPresent is true.
- Check params include trigger jsx-3, popup jsx-4, option jsx-5, and control state open.
- Per-check seven-layer traceability is preserved.
- Source artifact paths and hashes are present.
- generationPolicy explicitly prohibits Bob, test generation, patching, UI rendering, and compliance claims.

- Manual review, report boundary, and Bob constraints are preserved.
- Hash is stable with fixed createdAt.
- Hash changes when binding, check description, component path, or source artifact hash changes.
- Blocked Phase 2 or skipped Phase 3 produces skipped.
- Missing manifestUsed, trigger, popup, options, manual review, report boundary, Bob constraints, or requirements produces failed.

```
npm.cmd run typecheck
npm.cmd run test:analysis
npm.cmd run test:classification
npm.cmd run test:manifest
npm.cmd run test:spec
npm.cmd run specfirst:freeze-spec -- <run-id>
```

15. Handoff to Phase 5

Phase 5 should treat lockedSpec.json as read-only input. It should generate tests from the checks, not reinterpret rules or reload the manifest.

Phase 5 can rely on:

- lockedSpec.status is locked.
- nextPhase.canContinue is true.
- checks contain all required MVP test strategies.
- componentScope.selector identifies the test wrapper.
- testHarnessHints provide label, options, and onChange-spy needs.
- focusManagementStrategy indicates aria-activedescendant behavior.
- integrity.hash can be copied into the generated test header.
- bobPatchConstraints will later be used by the Bob prompt generator.

16. Definition of done

- Reads and validates all four Phase 1 to Phase 3 artifacts.
- Uses manifestUsed.json as the rulebook snapshot.
- Writes only lockedSpec.json.
- Produces locked, skipped, and failed outputs correctly.
- Generates all 11 MVP checks with component-specific params.
- Preserves seven-layer traceability per check.
- Includes source artifact hashes, generation policy, component bindings, component scope, test harness hints, manual review items, Bob constraints, accessibility claim boundary, and phase state boundary.
- Computes a stable SHA-256 hash with documented canonicalization.
- Does not call Bob, generate tests, run tests, patch code, render UI, or claim compliance.

17. Final implementation note

Phase 4 is a trust boundary, not a feature demo. Its value is that Bob will later patch against a contract that Bob did not create and cannot move. This is the mechanism that turns Bob from a free-form coding assistant into a governed remediation engine.